

Semana 13 — Áudio (AudioStreamPlayer), Profiling e Optimization

Semana 13

Áudio (AudioStreamPlayer), Profiling e Optimization

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade IV — Produzir como um Pequeno Estúdio (Semanas 12–14)

Projeto: Vertical Slice *O Templo Esquecido*

Desafio Técnico — Rubrica 2, com Feedback Formal (não encerra a Unidade IV)

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender `AudioStreamPlayer` como resposta a eventos de gameplay, não como trilha desconectada da ação
- Integrar sons a eventos já existentes no projeto (interação, passos, ambiente), reutilizando Signals e Components de módulos anteriores
- Compreender Profiling e Optimization como etapa obrigatória de produção, não ajuste isolado de última hora
- Usar o Profiler/Debugger nativo do Godot para identificar gargalos reais antes de qualquer otimização
- Propor e justificar, com solução própria, a otimização de um aspecto identificado no profiling do próprio Vertical Slice

Semana 13 — Áudio (AudioStreamPlayer), Profiling e Optimization

ENCONTRO 1

Áudio como Resposta a Eventos de Gameplay

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão do Encontro 2 da Semana 12 (materiais refatorados, zona externa composta, Code Review) (15 min)
- Introdução: áudio como resposta a eventos de gameplay, não como trilha isolada (20 min)
- Demonstração: `AudioStreamPlayer` reagindo a um Signal de interação já existente e a um evento de passos (30 min)
- Laboratório: cada grupo integra som a interação, passos e ambiente no próprio Vertical Slice (50 min)
- Feedback e fechamento (20 min)



Por que tratar áudio como um sistema à parte, desconectado dos sinais que o jogo já emite?

Pense no mesmo Signal que a Porta já usa desde a Semana 5.

O Problema: Áudio Desconectado do Gameplay

- Música/som de fundo tocado sem relação com o que acontece na tela não comunica nada ao jogador
- Áudio, em qualquer engine, deveria ser uma resposta a um evento que o gameplay já dispara
- O projeto já emite os sinais certos desde a Semana 5 (interação) e a Semana 2 (movimentação)

AudioStreamPlayer , **AudioStreamPlayer2D** , **AudioStreamPlayer3D**

- **AudioStreamPlayer** — som não posicional (ex.: UI, música)
- **AudioStreamPlayer2D** / **AudioStreamPlayer3D** — som posicional, com atenuação por distância
- Pergunta-chave para escolher o tipo certo: "esse som deveria mudar de volume se eu me afastar?"

Resposta Sonora a Eventos no Godot × Unity

Godot

- `AudioStreamPlayer` / `2D` / `3D` — três classes distintas conforme posicionamento
- Reage a Signals já existentes no projeto (`interacted`, eventos de movimentação)

Unity

- `AudioSource` único, configurado por Spatial Blend (2D/3D)
- Reage a `UnityEvent` / C# Actions ou chamada direta de script

Demonstração — AudioStreamPlayer Reagindo a um Signal

O que será construído:

- Um `AudioStreamPlayer` conectado ao Signal `interacted` já existente (Porta ou Alavanca)
- Um segundo exemplo de som não posicional versus posicional aplicado aos passos do Player

Por quê: mostrar que nenhum sistema novo de comunicação é introduzido — apenas um novo "ouvinte" de um sinal já emitido.

Imagem sugerida

*Objetivo: mostrar o **AudioStreamPlayer** como um novo "ouvinte" do mesmo Signal **interacted** que já aciona a reação da Porta.*

*Enquadramento: diagrama simples com a Porta ao centro emitindo o sinal **interacted**, e duas setas saindo dele — uma para a animação/abertura da Porta (já existente), outra para o novo **AudioStreamPlayer**.*

Elementos presentes: ícone de Porta, ícone de Signal, ícone de alto-falante conectado ao mesmo sinal.

*Destaque visual: a seta para o **AudioStreamPlayer** em destaque (cor diferente), reforçando que é a única novidade da semana.*

Legenda sugerida: "O AudioStreamPlayer não cria um novo canal de comunicação — apenas reage a um Signal que o projeto já emite desde a Semana 5."

Laboratório — Integração de Áudio ao Vertical Slice

Cada grupo, no próprio projeto:

1. Integra som a ao menos um evento de interação já existente (Signal `interacted`)
2. Integra som aos passos do Player, escolhendo entre `AudioStreamPlayer` e `AudioStreamPlayer2D/3D`
3. Integra um som ambiente à zona externa ou interna, com liberdade de escolha entre os sons disponíveis

Fechamento — Encontro 1

- Som integrado a pelo menos um evento de interação, aos passos do Player e a um elemento ambiente
- Nenhum sistema novo de comunicação entre objetos — apenas um novo "ouvinte" dos Signals já existentes
- Nenhuma alteração de mecânica ou sistema de gameplay já validado
- Próximo passo: Profiling e Optimization, no Encontro 2

Semana 13 — Áudio (AudioStreamPlayer), Profiling e Optimization

ENCONTRO 2

Profiling e Optimization

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (áudio integrado a interação, passos e ambiente) (15 min)
- Demonstração: leitura do Profiler/Debugger em um caso de gargalo conhecido pelo professor (25 min)
- Laboratório: cada grupo faz o profiling do próprio Vertical Slice e implementa uma otimização a partir do diagnóstico (55 min)
- Feedback formal sobre as otimizações realizadas — Desafio Técnico, Rubrica 2 (40 min)

Semana 13 — Áudio (AudioStreamPlayer), Profiling e Optimization



Como saber o que otimizar antes de otimizar qualquer coisa?

Pense nos Code Reviews já feitos nas Semanas 7, 10 e 12 — o mesmo princípio se aplica agora ao desempenho.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O Problema: Otimizar sem Medir

- Otimizar por suposição arrisca corrigir um problema que não é o gargalo real do projeto
- Profiling é o mesmo princípio de diagnóstico já praticado nos Code Reviews do semestre, aplicado agora ao desempenho
- O Vertical Slice acumula decisões desde o Módulo 1 (materiais, animação, IA, composição de cena) — o profiling é também uma releitura crítica de tudo isso

Profiler/Debugger do Godot

- Monitor de desempenho (FPS, tempo de frame)
- Monitores de memória, física e renderização
- Identificação de picos de custo por categoria, cruzados com decisões já tomadas em semanas anteriores

Profiling no Godot × Unity

Godot

- Profiler/Debugger nativo — monitores essenciais compactos (FPS, memória, física, renderização)
- Exige cruzar dados manualmente com mais frequência

Unity

- Unity Profiler — módulos mais granulares por sistema (Rendering, Audio, UI em painéis próprios)
- Mesma leitura de gargalos antes de qualquer decisão de otimização

Demonstração — Leitura do Profiler em um Gargalo Conhecido

O que será construído:

- Leitura guiada do Profiler/Debugger sobre o projeto de referência do professor, com um gargalo já conhecido de antemão (sem revelar a causa antes do laboratório)

Por quê: fixar o processo de leitura dos monitores antes de cada grupo aplicar o mesmo processo ao próprio projeto.

Imagem sugerida

Objetivo: mostrar a leitura dos monitores do Profiler/Debugger do Godot antes de qualquer decisão de otimização.

Enquadramento: captura conceitual do painel do Profiler, com destaque para os monitores de FPS/tempo de frame, memória, física e renderização lado a lado.

Elementos presentes: gráfico de tempo de frame com um pico visível, indicadores numéricos de memória e física, ícone de lupa sobre o pico de custo.

Destaque visual: o pico de custo destacado em vermelho, com uma seta apontando para a categoria responsável.

Legenda sugerida: "Medir antes de otimizar — o Profiler revela onde o tempo de frame e a memória do projeto estão sendo gastos."

Laboratório — Profiling e Otimização do Próprio Projeto

Cada grupo, no próprio Vertical Slice:

1. Abre o Profiler/Debugger e identifica ao menos um ponto real de custo elevado (geometria, materiais, iluminação, IA ou lógica de script/Orchestration)
2. Propõe uma otimização específica para esse ponto, com liberdade total de escolha entre os gargalos identificados
3. Implementa a otimização e confirma que nenhuma mecânica ou sistema já validado sofreu regressão

Arquitetura — Áudio e Diagnóstico de Desempenho

- Áudio (Encontro 1) → novo "ouvinte" dos Signals já existentes, sem novo canal de comunicação
- Profiling (Encontro 2) → leitura de todo o Vertical Slice acumulado desde o Módulo 1, sob a ótica de desempenho
- Otimização → correção pontual e justificada de um gargalo real, sem alterar mecânica ou geometria



Desafio Técnico — Otimização a partir do Profiling

Identifique, no próprio Vertical Slice, ao menos um gargalo real via Profiler/Debugger e implemente uma otimização justificada pelos dados observados.

OBJETIVOS

Entrega: Feedback Formal sobre as otimizações realizadas (Desafio Técnico — Rubrica 2): solução proposta, uso correto dos recursos do Godot, criatividade, organização e funcionamento.

Boas Práticas — Áudio e Desempenho

- Áudio sempre como reação a um Signal já existente, nunca como sistema com lógica própria
- Nenhuma otimização sem consulta prévia ao Profiler/Debugger
- Priorizar o gargalo de maior impacto real, não o mais fácil de corrigir
- Otimização como equilíbrio entre desempenho e resultado já validado em Playtests anteriores

Fechamento — Encontro 2

- Diagnóstico de profiling do próprio projeto, com ao menos um gargalo real identificado
- Otimização implementada e funcional para esse gargalo, sem regressão em nenhuma mecânica
- Feedback Formal sobre as otimizações realizadas (Rubrica 2) concluído
- Próximo passo: exportação do build final do Vertical Slice, na Semana 14

Resultado Esperado da Semana

- Vertical Slice com som integrado às principais ações de gameplay (interação, passos, ambiente) via `AudioStreamPlayer`
- Ao menos um gargalo de desempenho identificado por profiling e corrigido por uma otimização própria, justificada tecnicamente
- Nenhuma alteração de mecânica, sistema de gameplay ou geometria do Vertical Slice
- Projeto tecnicamente pronto, em som e desempenho, para a exportação final da Semana 14

Checklist da Semana

- [] Som integrado a pelo menos um evento de interação, aos passos do Player e a um elemento ambiente
- [] Nenhum sistema novo de comunicação — áudio reage a Signals já existentes
- [] Diagnóstico de profiling com ao menos um gargalo real identificado
- [] Otimização implementada e funcional, sem regressão em nenhuma mecânica
- [] Feedback Formal sobre as otimizações realizadas (Rubrica 2) concluído

Próximos Passos — Semana 14

A Semana 14 encerra a Unidade IV e o Módulo 4 com a exportação do build final do Vertical Slice — os ajustes de áudio e as otimizações realizadas nesta semana são parte direta do que será validado no Playtest cruzado entre grupos e no Code Review de encerramento, sem retrabalho de materiais, foliage, áudio ou otimização além de ajustes pontuais identificados nesse Playtest.

Leitura recomendada: Godot Docs — Audio; Godot Docs — Optimization (Profiler/Debugger); Unity Manual (consulta comparativa) — Audio Overview, Profiler; Kenney Assets (kenney.nl).